

08-Python 小项目实践

学习了 Python 基础语法后，要通过小项目把知识连起来。项目不一定复杂，但必须经历完整过程：理解任务、设计步骤、编写代码、测试、修改、整理提交。

一、学习目标

学完本节，你应该能做到：

1. 按项目方式组织 Python 文件。
 2. 在写代码前完成 IPO 分析。
 3. 从最小版本开始逐步增加功能。
 4. 使用变量、条件、循环、列表和函数完成小程序。
 5. 准备正常数据、边界数据和特殊数据进行测试。
 6. 写简单说明和调试记录。
-

二、Python 项目文件夹规范

每个项目单独建立文件夹。

示例：

```
Python练习
├── 成绩等级判断器
│   ├── main.py
│   ├── 说明.txt
│   ├── 测试记录.txt
│   └── 修改记录.txt
```

文件说明

文件	作用
main.py	主程序代码
说明.txt	写清项目功能和运行方法
测试记录.txt	记录测试数据和结果
修改记录.txt	记录遇到的问题和解决方法

如果项目有多个版本，可以使用：

```
main-v1.py
main-v2.py
```

但提交最终作业时，最好保留一个清楚的入口文件，例如 `main.py`。

三、项目 1：成绩等级判断器

1. 任务说明

输入一个 0 到 100 的成绩，输出对应等级。

规则：

分数范围	等级
------	----

90 到 100	优秀
----------	----

80 到 89	良好
---------	----

60 到 79	合格
---------	----

0 到 59	需要努力
--------	------

2. IPO 分析

项目	内容
----	----

输入	一个成绩 <code>score</code>
----	-------------------------

处理	按分数范围判断等级
----	-----------

输出	优秀、良好、合格或需要努力
----	---------------

3. 自然语言步骤

1. 输入成绩
2. 把输入转换成整数
3. 如果成绩 ≥ 90 ，输出优秀
4. 否则如果成绩 ≥ 80 ，输出良好
5. 否则如果成绩 ≥ 60 ，输出合格
6. 否则输出需要努力

4. 代码

```
score = int(input("请输入成绩："))

if score >= 90:
    print("优秀")
elif score >= 80:
```

```
    print("良好")
elif score >= 60:
    print("合格")
else:
    print("需要努力")
```

5. 测试记录

测试数据 预期结果 实际结果 是否通过

100	优秀		
90	优秀		
89	良好		
80	良好		
79	合格		
60	合格		
59	需要努力		
0	需要努力		

6. 拓展功能

增加输入范围检查：

```
score = int(input("请输入成绩："))

if score < 0 or score > 100:
    print("成绩应在 0 到 100 之间")
elif score >= 90:
    print("优秀")
elif score >= 80:
    print("良好")
elif score >= 60:
    print("合格")
else:
    print("需要努力")
```

四、项目 2：奇偶数判断器

1. 任务说明

输入一个整数，判断它是奇数还是偶数。

2. IPO 分析

项目	内容
----	----

输入 一个整数 number

处理 判断 number 除以 2 的余数是否为 0

输出 偶数或奇数

3. 关键知识

% 是取余运算。

```
print(7 % 2)
```

结果是：

1

因为 7 除以 2 余 1。

4. 代码

```
number = int(input("请输入一个整数："))

if number % 2 == 0:
    print("这是偶数")
else:
    print("这是奇数")
```

5. 测试数据

输入 预期结果

0 偶数

1 奇数

2 偶数

99 奇数

100 偶数

6. 拓展

增加判断正数、负数或零：

```
number = int(input("请输入一个整数："))

if number > 0:
    print("这是正数")
elif number < 0:
```

```
        print("这是负数")
    else:
        print("这是零")

    if number % 2 == 0:
        print("这是偶数")
    else:
        print("这是奇数")
```

五、项目 3：简易计算器

1. 任务说明

输入两个数字和一个运算符，输出计算结果。

支持：

- 加法 +
- 减法 -
- 乘法 *
- 除法 /

2. IPO 分析

项目	内容
输入	第一个数、第二个数、运算符
处理	根据运算符选择计算方式
输出	计算结果或错误提示

3. 最小版本代码

```
a = float(input("请输入第一个数: "))
b = float(input("请输入第二个数: "))
op = input("请输入运算符 (+ - * /): ")

if op == "+":
    print("结果是", a + b)
elif op == "-":
    print("结果是", a - b)
elif op == "*":
    print("结果是", a * b)
elif op == "/":
    print("结果是", a / b)
else:
    print("不支持这个运算符")
```

4. 增加除数检查

如果除以 0，程序会出错。可以增加判断：

```
a = float(input("请输入第一个数: "))
b = float(input("请输入第二个数: "))
op = input("请输入运算符 (+ - * /): ")

if op == "+":
    print("结果是", a + b)
elif op == "-":
    print("结果是", a - b)
elif op == "*":
    print("结果是", a * b)
elif op == "/":
    if b == 0:
        print("除数不能为 0")
    else:
        print("结果是", a / b)
else:
    print("不支持这个运算符")
```

5. 测试数据

第一个数	第二个数	运算符	预期结果
3	5	+	8
10	4	-	6
6	7	*	42
8	2	/	4
8	0	/	除数不能为 0
8	2	%	不支持这个运算符

六、项目 4：猜数字游戏

1. 任务说明

程序设置一个秘密数字，用户不断猜，直到猜对。

2. 最小版本

```
secret = 7
guess = int(input("请猜一个 1 到 10 的数字: "))

while guess != secret:
    print("猜错了，再试一次。")
    guess = int(input("请猜一个 1 到 10 的数字: "))
```

```
print("猜对了! ")
```

3. 增加大小提示

```
secret = 7
guess = int(input("请猜一个 1 到 10 的数字: "))

while guess != secret:
    if guess > secret:
        print("太大了")
    else:
        print("太小了")
    guess = int(input("请再猜一次: "))

print("猜对了! ")
```

4. 增加次数统计

```
secret = 7
count = 1
guess = int(input("请猜一个 1 到 10 的数字: "))

while guess != secret:
    if guess > secret:
        print("太大了")
    else:
        print("太小了")
    count = count + 1
    guess = int(input("请再猜一次: "))

print("猜对了! ")
print("你一共猜了", count, "次")
```

5. 拓展：随机数字

如果已经学习了导入模块，可以使用随机数：

```
import random

secret = random.randint(1, 10)
count = 1
guess = int(input("请猜一个 1 到 10 的数字: "))

while guess != secret:
    if guess > secret:
        print("太大了")
    else:
        print("太小了")
    count = count + 1
    guess = int(input("请再猜一次: "))
```

```
print("猜对了! ")
print("你一共猜了", count, "次")
```

七、项目 5：问答计分系统

1. 任务说明

程序提出问题，用户回答，答对加分，最后显示总分。

2. 最小版本

```
score = 0

answer = input("问题 1: 2 + 3 = ")
if answer == "5":
    print("答对了")
    score = score + 1
else:
    print("答错了")

answer = input("问题 2: 中国的首都是哪里? ")
if answer == "北京":
    print("答对了")
    score = score + 1
else:
    print("答错了")

print("你的总分是", score)
```

3. 使用列表改进

```
questions = ["2 + 3 = ", "中国的首都是哪里? ", "Python 中输出用哪个函数? "]
answers = ["5", "北京", "print"]
score = 0

for i in range(len(questions)):
    user_answer = input(questions[i])
    if user_answer == answers[i]:
        print("答对了")
        score = score + 1
    else:
        print("答错了, 正确答案是", answers[i])

print("你的总分是", score)
```

4. 思考

1. questions 和 answers 的顺序为什么必须对应?

2. 如果增加第 4 个问题，要改哪里？
 3. 用户输入有空格或大小写不同，会不会影响判断？
-

八、项目 6：班级兴趣统计

1. 任务说明

输入多名同学的兴趣，统计并输出结果。

2. 简单版本

```
hobbies = []

for i in range(5):
    hobby = input("请输入一名同学的兴趣：")
    hobbies.append(hobby)

print("收集到的兴趣有：")
for hobby in hobbies:
    print(hobby)
```

3. 统计某个兴趣出现次数

```
hobbies = []

for i in range(5):
    hobby = input("请输入一名同学的兴趣：")
    hobbies.append(hobby)

target = input("请输入要统计的兴趣：")
count = 0

for hobby in hobbies:
    if hobby == target:
        count = count + 1

print(target, "出现了", count, "次")
```

4. 拓展方向

1. 输入人数由用户决定。
 2. 输出所有兴趣列表。
 3. 统计最受欢迎兴趣。
 4. 把结果写成一段总结文字。
-

九、项目开发五步法

每做一个项目都按下面流程。

第 1 步：写任务说明

示例：

项目名称：猜数字游戏

项目功能：用户猜 1 到 10 的数字，程序提示太大或太小，直到猜对。

第 2 步：写 IPO 表

项目	内容
输入	用户猜的数字
处理	与秘密数字比较
输出	太大、太小或猜对

第 3 步：写最小版本

先完成能运行的最小功能，不要一开始就加很多花样。

第 4 步：测试和修改

至少准备 3 组测试：

测试类型	例子
正常数据	猜对或猜错
边界数据	1、10
特殊情况	连续猜错多次

第 5 步：整理提交

提交前检查：

- ☐ 项目文件夹命名清楚。
- ☐ 主程序文件存在。
- ☐ 说明文件写清运行方法。
- ☐ 测试记录至少 3 条。
- ☐ 修改记录至少 1 条。
- ☐ 压缩整个项目文件夹。

十、项目说明模板

可以复制下面模板到 说明.txt。

项目名称：

作者：

一、项目功能

这个程序可以.....

二、运行方法

1. 打开 main.py

2. 点击运行

3. 根据提示输入数据

三、用到的知识

变量：

条件判断：

循环：

列表或函数：

四、测试数据

1.

2.

3.

五、遇到的问题和解决方法

问题：

原因：

解决方法：

六、还可以改进的地方

十一、本节达标清单

- ☐ 我能为每个 Python 项目建立单独文件夹。
- ☐ 我能写项目说明、测试记录和修改记录。
- ☐ 我能用 IPO 表分析项目。
- ☐ 我能从最小版本开始逐步增加功能。
- ☐ 我完成了至少 2 个 Python 小项目。
- ☐ 我能使用条件判断处理分类问题。
- ☐ 我能使用循环处理重复任务。
- ☐ 我能使用列表保存多个数据。
- ☐ 我能用多组测试数据检查程序。
- ☐ 我能向别人说明自己的项目如何运行。